ESP32-S3 + ADXL355

$\sim$43 EUR of parts, and one thing it cannot do

MCU	ESP32-S3, 2$\times$LX7 @ 240 MHz
Sensor	ADXL355: **1 kHz**, 25 μg/$\sqrt{\text{Hz}}$, 20-bit
Alternative	IIS3DWB profile, build flag
Acquisition	4096 @ 4 kHz (1.02 s), **0.98 Hz**
Radio	Wi-Fi + BLE 5
Battery life	2.5 y at 6 h — or use mains

Branch `hw/esp32s3-adxl355`.

Never flashed. Written against ESP-IDF v5.2; the DSP above the board driver is verified by the shared pipeline's host tests.

The IIS3DWB acquisition path deliberately returns `ERR_NOT_READY` rather than falling through to the ADXL355 register map. A wrong register map produces plausible numbers; an explicit refusal does not.

The sensor choice is the architecture

	ADXL355	IIS3DWB
Usable BW	1 kHz	**6.3 kHz**
Noise	**25**	75
Resolution	**20-bit**	16-bit
Current	**200 μA**	1.1 mA

Bearing impacts ring the structure at 2–20 kHz. The ADXL355's anti-alias filter is at 1 kHz *by design*, so that band is removed before digitisation. There is no information left to extract.

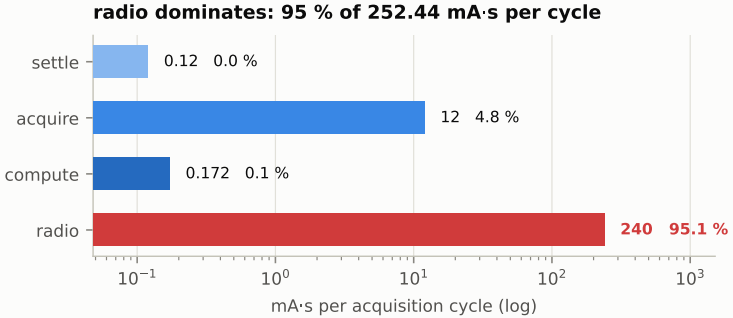
The config checker says so, unprompted, on every run:

```
fs 4000 Hz, n_fft 4096 (0.98 Hz), acq 4096 (1024 ms)
envelope:  decimate 2 -> 2000 Hz, 1024 pts (1.95 Hz),
           band 500-1000 Hz
NOTE the demodulation band (500-1000 Hz) is below
     the 2-20 kHz range where bearing impacts
     excite structural resonances.  Bearing
     evidence is weak; treat a negative
     result as 'not measured'
```

The easy alternative — one profile, a band straddling 1 kHz, and confident-looking bearing numbers — would produce work orders from filtered-out information. That defect is *undetectable from the output*, because every number is in range.

So what: It is not a worse sensor, it is a different one. 10 dB better at low frequency: slow machines, ISO velocity, unbalance, looseness.

Wi-Fi dictates the battery chemistry



240 mA·s per association — **1 400**× the sensing and compute combined. Nothing about the DSP, the sensor or the record length changes that number.

So: 6-hour default interval, a 4096-frame queue, a transmit path that *batches*, and deep sleep rather than light sleep (7 μA vs 240 μA — light sleep alone would be 40× the acquisition budget).

The chain nobody expects:

```
Wi-Fi 350 mA peaks
  -> rules out LiSOCl2 (pulse limit ~10s of mA)
     -> must use Li-ion (2.5 %/month = 72 uA)
        -> self-discharge > circuit draw above 1 h
           -> life saturates near 2.7 years
              -> honest answer:  mains or PoE
```

Compare the Nordic kit: LTE-M costs 95 mA·s — a *more* expensive radio — but its pulse profile permits a $LiSOCl_2$ D-cell at 0.08 %/month, giving **9.4 years**. Five times the life from the more expensive radio.

Two things that are correctness requirements, not optimisations

Dual-core

`CONFIG_FREERTOS_UNICORE=n`, and not for throughput.

The ADXL355 FIFO is 96 samples — **24 ms** at 4 kHz. The ESP-IDF Wi-Fi task outranks application tasks and can hold the CPU for tens of milliseconds during association.

On one core that overruns the FIFO, and the block becomes a concatenation of two different times: a spectrum with a step discontinuity that looks exactly like an impact train.

Three defences: acquisition pinned to core 1, the application never transmits during an acquisition, and the overrun flag refuses the block rather than repairing it.

The sequence number in RTC memory

Deep sleep restarts from `app_main` with SRAM gone. Ingest deduplicates on `(device_id, sequence)` so that store-and-forward replays are idempotent.

A node that restarts its counter at zero therefore has **every frame after the first wake silently discarded as a duplicate**. It looks healthy and transmits nothing — the worst available failure mode, because it is invisible from both ends.

`pdm_app_restore_sequence` was added to the shared core for exactly this, and the first-boot checklist in `tools/flash.sh` says to verify that the second wake emits sequence 1.

The kit's worst spec is also its best

The ADXL355's output rate comes from an internal oscillator with no tolerance better than ±**5 %** over temperature — worse than the IIS3DWB's ±3 %.

But this board has a ±**10 ppm crystal** and `esp_timer`, so the true elapsed time of an acquisition is measurable to microseconds. `samples / elapsed` *is* the real rate.

The STWIN.box cannot do this: its FIFO carries no timestamps at 26.667 kHz and there is no reference clock path into the sensor. Its fix needs a reference tone through the self-test actuator.

Why it matters. A 3 % rate error moves a 106 Hz BPFO line by 3.2 Hz. By the fifth harmonic that is 16 Hz against a ±10.6 Hz matching window, so the top harmonics fall outside and `n_matched` under-reports exactly when the evidence is strongest.

Status: identified, not implemented. `pdm_board_acquire` reports the nominal rate with a comment saying so. It is the cheapest accuracy win anywhere in this repository and about a dozen lines of code.

So what: The niche: brownfield retrofit, high asset count, a socket or PoE drop nearby. On battery, the ST and Nordic kits are better at their own jobs.